Healthcare Planning Assistant Agent

Project Report

Prepared according to the Medicaps University - Datagami Skill Based Course report format

Course Name	Agentic AI
Institution Name	Medicaps University - Datagami Skill Based Course
Group Name	Placed BtechGroup07
Project Number	AAI-02
Academic Year	2025-26

Student Name(s) & Enrolment Number(s)

Sr. No.	Student Name	Enrolment Number
1	CHETAN OSWAL	EN22CS301295
2	IRYA PATNI	EN22CS301436
3	UDAY DUBEY	EN22EL301058
4	HIMANI JAISWAL	EN22CS301423

Industry Mentor Name	
University Mentor Name	

Report Overview

This report presents the design, implementation approach, architecture, workflow, and expected outputs of the Healthcare Planning Assistant Agent. The document is prepared using the provided project report format, the submitted High Level Design, the submitted Low Level Design, and the repository material. It explains both the academic intent of the project and the technical logic used to build the planning assistant.

The project focuses on using Agentic AI to support healthcare workflow planning. Instead of asking a user to manually organize treatment-related steps, the system accepts a high-level goal, decomposes it into smaller tasks, checks whether required resources are available, and prepares a dependency-aware execution schedule. This reduces manual effort and demonstrates how AI systems can support structured operational planning in a healthcare context.

Document Structure

- Problem statement, objectives, and project scope.
- Proposed solution including features, architecture, and technical stack.
- Detailed explanation of high-level and low-level design decisions.
- Workflow, module design, process flow, and API-level understanding.
- Results, expected outputs, key outcomes, limitations, and enhancements.

Problem Statement & Objectives

1. Problem Statement

Healthcare planning often involves coordination among doctors, rooms, equipment, and time-bound treatment steps. In real environments, these planning activities are frequently performed manually or through fragmented systems. This can result in scheduling conflicts, inconsistent resource use, delay in care delivery, and difficulty in adapting when a required resource is unavailable.

The Healthcare Planning Assistant Agent addresses this problem by introducing an Agentic AI based planning system. The system allows a user to enter a high-level healthcare objective such as treatment planning or care coordination. It then interprets the objective, converts it into actionable tasks, validates resource availability, and returns a structured schedule that respects task dependencies. The aim is not to replace medical decision-making, but to improve operational planning and workflow orchestration.

2. Project Objectives

- To design an AI-driven healthcare planning assistant that can work from natural-language goals.
- To decompose a broad healthcare objective into smaller and logically ordered tasks.
- To incorporate resource validation so that the plan reflects practical feasibility.
- To generate a dependency-aware schedule that improves clarity and execution flow.
- To build a simple user-facing interface that makes the planning system accessible and easy to test.
- To demonstrate the use of CrewAI, LangChain, Gemini, Python, and Streamlit in a unified academic project.

3. Scope of the Project

The current scope of the project is a functional prototype of a healthcare workflow planning system. It accepts user input through a Streamlit frontend, performs reasoning using an agent configured with Gemini through LangChain, and validates resource needs with a mock tool. The final output is a structured plan containing tasks, dependencies, and an execution schedule.

The project scope mainly covers workflow generation and orchestration. It does not attempt to provide real clinical diagnosis, access real hospital databases, or process sensitive patient records. The current design is intentionally modular so that real APIs, hospital systems, or patient databases can be added later without redesigning the full application.

Need and Relevance of the Project

The healthcare domain contains many interdependent activities where timing and coordination matter. Even a simple treatment workflow may require consultation, diagnostics, doctor availability, room allocation, and equipment readiness. A planning system that can reason over such dependencies has value in training, prototyping, simulation, and future operational decision support.

This project is relevant because it demonstrates how Agentic AI can be applied beyond chat-based responses. It shows that an AI agent can take a goal, reason about intermediate steps, use tools, and return a structured operational output. That makes the project academically strong and practically extensible.

Proposed Solution

1. Key Features

- Natural-language goal input through a clean Streamlit interface.
- Planner Agent that interprets the goal and breaks it into actionable steps.
- Resource availability checking for doctors, rooms, and equipment using a tool-based approach.
- Dependency-aware task ordering and execution logic.
- Optimized schedule generation for the final workflow output.
- Human-readable result presentation in the interface.
- Modular architecture that supports future expansion with additional tools, APIs, and data sources.

2. Overall Architecture / Workflow

The system follows a layered and modular architecture. The presentation layer is implemented using Streamlit and is responsible for collecting the healthcare goal from the user and displaying the generated plan. The application layer contains the Planner Agent, which is the core decision-making unit. The service layer includes resource-checking logic and execution support. The integration layer connects the system to the Gemini model through LangChain.

At runtime, the user enters a high-level healthcare goal. The system passes this goal to the Planner Agent, which interprets the request and breaks it into smaller tasks. For each major task, the agent can call a resource assessment tool to confirm whether important resources are available. Once dependencies and feasibility are known, the execution logic produces an ordered and optimized schedule. This final plan is then shown back to the user through the frontend.

3. Tools & Technologies Used

Technology / Tool	Purpose in Project
Python 3.10+	Primary programming language used for backend implementation and integration.
CrewAI	Used to define the Planner Agent, tasks, and agent workflow execution.
LangChain	Used as the integration layer for LLM interaction and reasoning flow management.
Google Gemini API	Provides large language model capabilities for understanding goals and generating task plans.
Streamlit	Used to build the interactive frontend and display outputs to the user.
Git and GitHub	Used for version control and project repository hosting.
.env configuration	Used for secure handling of API keys and environment variables.

High Level Design Analysis

The submitted High Level Design describes the project as an AI-powered healthcare workflow planning assistant. The HLD shows how the user interface, backend orchestration, planner agent, and resource tool work together to convert an abstract healthcare goal into a usable action plan.

User Interface - Streamlit

The frontend acts as the entry point for the user. It is responsible for loading environment settings, collecting the healthcare goal, initiating the planning sequence, and presenting the returned output in a readable form.

From a design perspective, the UI is intentionally simple. This is important because the complexity of the project lies in planning logic rather than frontend interaction. A minimal interface also makes testing easier and improves usability for academic demonstration.

Core Agent Backend

The backend acts as the integration wrapper that connects the frontend to the planning logic. It compiles the configured agent and tasks into a CrewAI workflow and manages execution using the API key supplied through environment variables.

This layer is important because it isolates orchestration logic from presentation. That separation improves maintainability and makes the system easier to extend.

Medical Planner Agent

The Planner Agent is the central intelligence of the system. It receives the high-level goal and applies reasoning to convert that goal into ordered, actionable tasks. The agent is designed specifically for hospital planning style scenarios, which means its outputs are not just generic text but operationally structured plans.

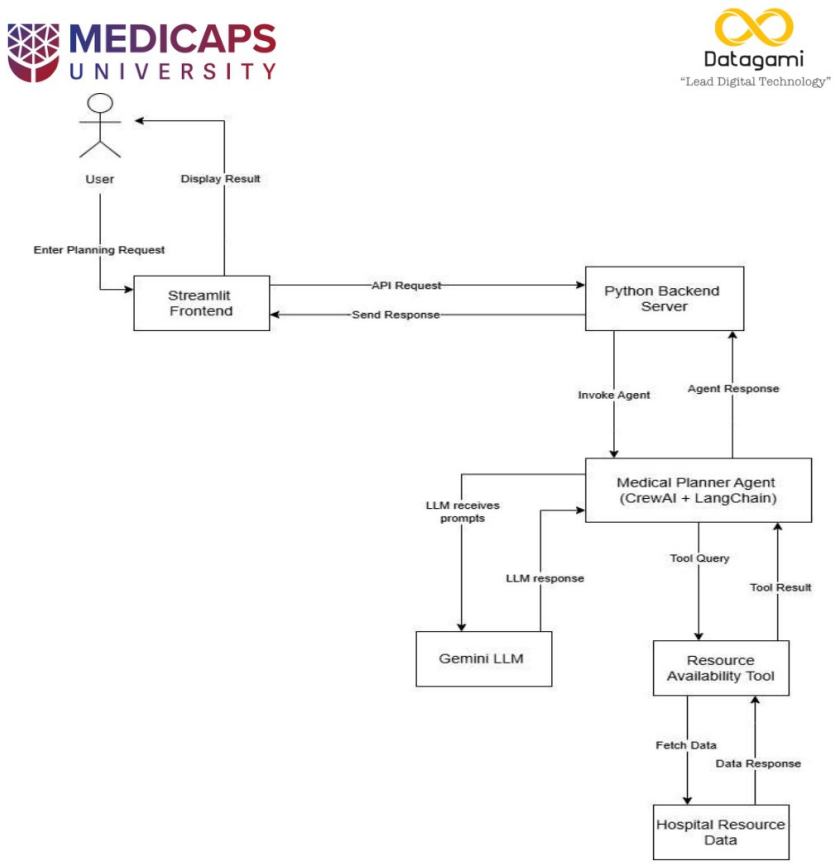
The agent works with the Gemini model and uses prompt design plus task logic to maintain dependencies and planning discipline.

Resource Assessment Interface Tool

This tool allows the agent to check whether required resources are available. In the current implementation, the tool uses mock logic rather than live hospital systems, but the pattern is realistic because the agent can still make tool calls and revise its planning output based on the returned availability information.

That makes the HLD valuable because it shows how tool usage can be built into an agentic workflow instead of relying only on plain language generation.

Figure 1. High level architecture from the submitted design.



6. Application Design

The application is designed as a simple web-based system.

It includes:

- A user interface built using Streamlit where users can enter their medical planning request.
- A backend system built using Python to handle processing and system logic
- Integration with the Gemini API to support AI reasoning and planning
- A resource availability module that retrieves hospital resource information.

Structured prompts and agent-based reasoning ensure relevant medical plan generation.

Medicaps University – Datagami Skill Based Course – Project Report | 5

The figure illustrates the layered interaction between the interface, planner agent, service logic, and model integration.

Low Level Design Analysis

The Low Level Design expands the project into implementable modules and operational flows. It explains how the system is divided into components, how information moves through the system, and how the planner, resource checker, execution engine, and LLM integration interact in detail.

The LLD is especially important for this project because the value of the solution depends on clear internal coordination. A high-level goal alone is not enough; the system needs module-level clarity so that planning, validation, scheduling, and output generation happen in a predictable sequence.

Component Design

- User Interface Component: accepts goal input, triggers the backend, and displays outputs.
- Planner Agent Component: interprets the goal, breaks it into smaller tasks, and coordinates planning logic.
- Resource Availability Component: simulates practical checks for doctors, rooms, and equipment.
- Execution Engine: resolves dependencies, allocates time, and generates the final schedule.
- LLM Integration Component: links the system to Gemini through LangChain for reasoning and output generation.

Key Design Considerations

- Scalability through modular components and add-on tools.
- Extensibility so that real APIs, patient systems, and hospital resources can be integrated later.
- Reliability by handling unavailable resources gracefully.
- Performance through controlled LLM usage and structured reasoning.
- Security through secure API key handling and non-exposure of sensitive patient data.
- Usability through minimal input requirements and clear result presentation.

API Catalogue in the LLD

- Initialize Planning: starts the planning sequence using the user goal.
- Generate Plan: creates the task list and step-level breakdown.
- Check Resource Availability: verifies whether required resources are available.
- Generate Schedule: returns the optimized task sequence and time allocation.

- Checks **resource availability** such as doctors and equipment
- Generates a **dependency-aware and optimized schedule**
- Displays the result through an interactive interface

Key capabilities include:

- Intelligent task decomposition
- Tool-based reasoning
- Dynamic decision-making
- Real-time user interaction

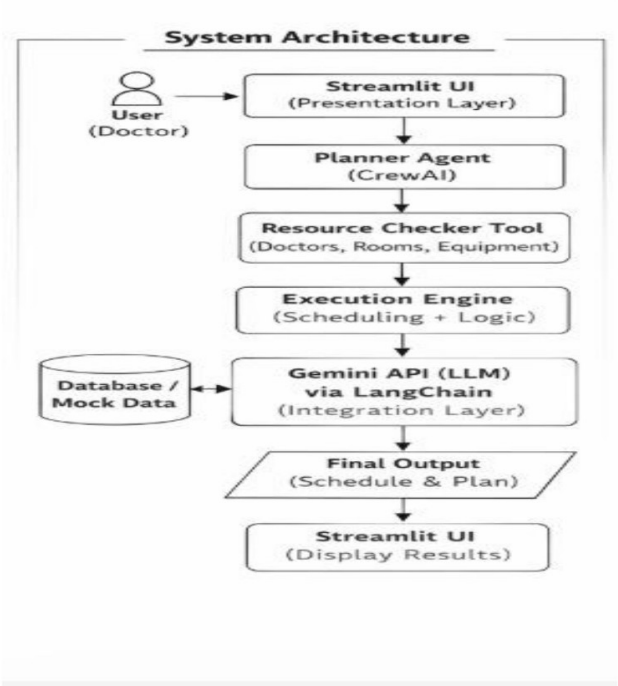


Figure 2. Low level system decomposition showing internal implementation-oriented modules.

Process Flow and Information Flow

Step-by-Step Process Flow

1. The user opens the Streamlit application.
2. The user enters a healthcare goal such as treatment planning or workflow coordination.
3. The interface sends the goal to the backend execution layer.
4. The Planner Agent interprets the goal and decomposes it into smaller tasks.
5. The agent identifies resource requirements for the generated tasks.
6. The resource tool checks availability and returns a status.
7. The execution logic resolves dependencies and assigns task order.
8. The system produces an optimized schedule with tasks and time allocation.
9. The final planning output is returned to the interface for display.

Information Flow

Information first enters the system as user input. This goal data is then transformed into task data by the Planner Agent. The generated tasks are passed to the resource tool for validation. Resource data is sent back to the planning logic so that any unavailable requirement can influence the final output. After dependency resolution and timeline generation, the processed data is returned to the user as a structured plan.

The LLD also identifies key data elements such as goal data, task data, and resource data. This is useful because it shows that the project is not just prompt-based but data-flow driven. The movement from input to processing, validation, optimization, and output gives the system a disciplined operational structure.

7. Process Flow

The process works in the following steps:

1. The user enters a medical planning request through the interface.
2. The backend system receives the request and processes the input
3. The request is passed to the medical planning agent for reasoning.
4. The agent sends a prompt to the Gemini API to analyze the request and generate planning logic.
5. The system checks available hospital resources through the resource availability tool.
6. Based on the reasoning and resource information, the system generates the final medical schedule or plan.
7. The final result is then displayed to the user through the interface.

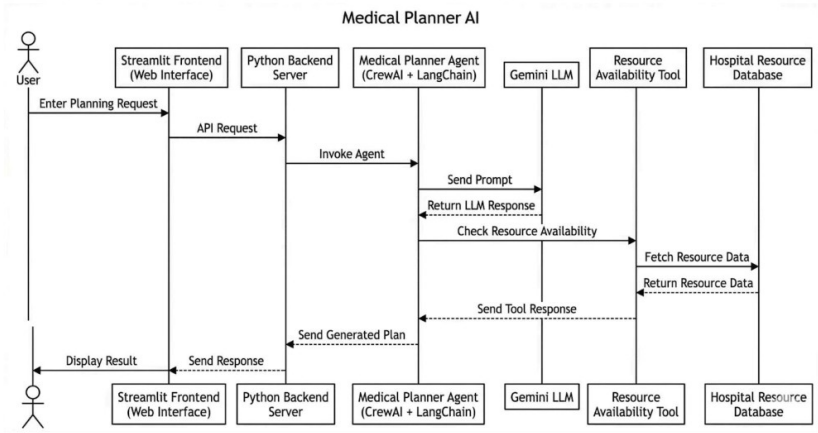
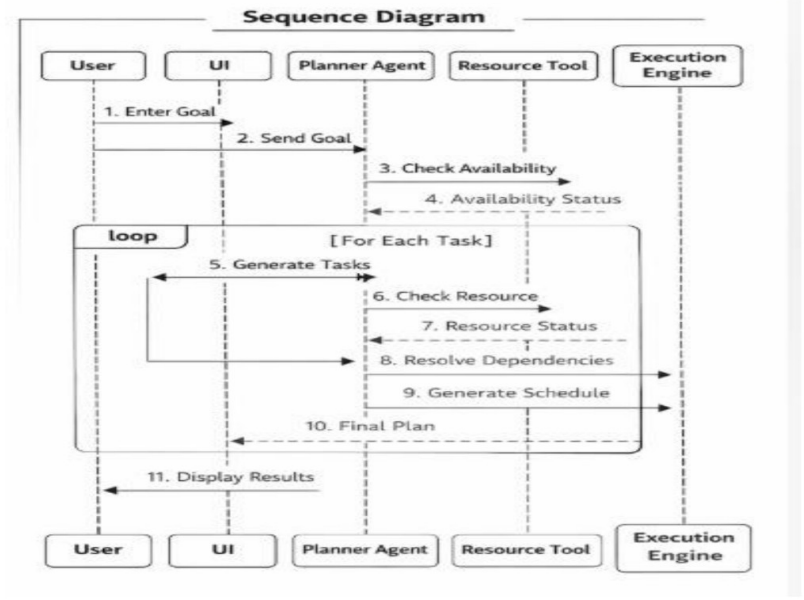


Figure 3. Process flow showing movement from user goal to planning output.



2.4 Components Design

The system is divided into multiple components, each with a specific responsibility.

2.4.1 User Interface Component

The UI is built using Streamlit and serves as the entry point for users.

Responsibilities:

- Accept user input
- Trigger backend processes
- Display results clearly

Key Features:

- Simple input field

Medicaps University-Datagami based skilled course-Project Report| 9

Figure 4. Sequence-level representation of internal execution logic.

Implementation Understanding Based on the Repository

The repository is organized into backend, frontend, and docs sections, which aligns with the architecture described in the submitted HLD and LLD. The README further explains that the system uses a single Planner Agent together with a custom resource-checking tool to create a detailed schedule from a high-level healthcare goal.

The implementation emphasizes a clean separation of roles. The frontend handles interaction, the backend coordinates the agent execution, the Planner Agent performs reasoning, and the resource tool simulates external operational checks. This confirms that the actual repository structure supports the design claims made in the HLD and LLD.

Repository-to-Design Mapping

Repository / Module	Design Role	Explanation
frontend / app.py	Presentation Layer	Collects healthcare goals from the user and displays planning output in the Streamlit interface.
backend or src/agent_backend.py	Application / orchestration layer	Builds and runs the CrewAI workflow and connects the model with the planning logic.
Planner Agent configuration	Decision-making unit	Interprets the goal, breaks it into tasks, and maintains dependency-aware planning.
CheckResourceAvailabilityTool	Service layer tool	Provides resource availability status so that the plan can reflect practical feasibility.
Gemini via LangChain	Integration layer	Supplies language understanding and reasoning support for plan generation.
docs	Project documentation	Contains supporting design or academic documentation for explaining the system.

Agent and Task Behaviour

The README and LLD indicate that the Planner Agent is configured as a medical planning coordinator. It is expected to reason over task order, dependencies, and schedule structure. The LLD also states that the task parameters impose a strict process: goal decomposition, resource identification, validation, and scheduling. This is important because it makes the system workflow-driven rather than producing unconstrained text.

The custom resource availability tool is implemented as a CrewAI tool. Although it currently uses predefined matching logic instead of live APIs, it demonstrates the correct agentic pattern: the model

can call a tool, receive structured feedback, and adjust the plan. That pattern forms the foundation for future real-world integrations.

Results & Output

1. Screenshots / Outputs

The provided designs and repository describe the expected project output as a structured healthcare workflow plan displayed through a Streamlit interface. The output is not merely a paragraph response. It is expected to contain a sequence of tasks, dependencies, resource checks, and a schedule that can be read and acted upon by the user.

A typical output begins with the entered healthcare goal, followed by a decomposition of that goal into actionable steps. The system then reflects whether required resources are available and finally produces an ordered execution schedule. This makes the output suitable for academic demonstrations of planning, orchestration, and AI-assisted workflow generation.

2. Reports / Models / Structured Outputs

Output Element	What the System Produces
Goal interpretation	Meaningful understanding of the user-entered healthcare objective.
Task breakdown	Smaller tasks generated from the original goal.
Dependency mapping	Logical relationship among tasks so that sequence is preserved.
Resource validation	Availability status of doctors, rooms, and equipment.
Execution schedule	Optimized order and timing allocation for plan execution.
Human-readable response	Final rendered output shown in the Streamlit interface.

3. Key Outcomes

- Demonstrates successful use of Agentic AI in a healthcare planning scenario.
- Shows how a Planner Agent can move from broad goals to actionable and dependency-aware outputs.
- Illustrates tool-augmented reasoning rather than plain text generation alone.
- Provides a modular foundation for future real-world healthcare system integration.
- Acts as a strong academic prototype for understanding planning agents, LLM integration, and workflow automation.

Illustrative Example of Expected Output

If the user enters a goal such as 'Treatment Planning for a Patient Requiring Consultation, MRI, and Follow-up', the system may produce a plan that begins with consultation scheduling, checks doctor and MRI room availability, places diagnostic steps in the correct order, and then generates a follow-up task sequence. If a required resource is unavailable, the schedule can be adjusted accordingly.

This expected behaviour is useful because it demonstrates not only natural-language understanding but also operational reasoning. The system therefore behaves more like a planning assistant than a standard chatbot.

Discussion, Strengths, and Limitations

Project Strengths

- Clear alignment between high-level design, low-level design, and repository implementation.
- Use of modular architecture, which improves maintainability and future extensibility.
- Good academic demonstration of agent, tool, and model integration.
- Simple interface that keeps the focus on planning intelligence.
- Feasible pathway for future conversion from mock resources to real APIs.

Current Limitations

The project currently uses a mock resource availability mechanism rather than a real hospital system. This means the prototype demonstrates planning logic well, but real-world accuracy would require live integrations, role-based permissions, and validated resource datasets.

The system is designed for workflow planning and scheduling support, not direct medical diagnosis or treatment recommendation. Any future production use would need strong governance, healthcare domain validation, auditability, and compliance controls.

At its current stage, the application appears best suited for academic demonstration, prototyping, and controlled planning simulation. However, that does not reduce its value. In fact, the current design is strong precisely because it creates a safe and modular foundation for future applied work.

Quality Assessment

From a project evaluation perspective, the system performs well on architectural clarity. The HLD establishes a clear purpose and layered structure. The LLD provides component detail, process flow, and API-level explanation. The repository reflects the same modular thinking. This consistency is a major strength in a student project report.

The project also shows good technology choices. CrewAI fits the planning-agent requirement, LangChain supports model integration, Gemini provides reasoning capability, and Streamlit keeps the interface lightweight. Together they form a practical and understandable stack for this type of problem.

Conclusion

The Healthcare Planning Assistant Agent is a well-structured Agentic AI project that demonstrates how healthcare workflow planning can be automated from a natural-language goal. The system accepts a user objective, decomposes it into actionable tasks, validates resource availability, resolves dependencies, and generates an optimized execution schedule. In doing so, it moves beyond conversational AI and enters the area of operational planning support.

The project successfully combines a clean Streamlit frontend with CrewAI-based orchestration, LangChain integration, Gemini reasoning, and a tool-based resource checking mechanism. The

submitted High Level Design and Low Level Design clearly explain the system architecture, module interactions, process flow, and implementation logic. Overall, the project provides a strong academic foundation and a credible prototype for future intelligent healthcare planning solutions.

A key learning from this project is that effective AI systems often depend on structured workflows rather than model output alone. By combining a planner, a tool, and execution logic, the system becomes more reliable, explainable, and extensible. This is the central value demonstrated by the Healthcare Planning Assistant Agent.

Future Scope & Enhancements

- Integration with real hospital management systems and scheduling APIs.
- Connection to patient databases with appropriate privacy and security controls.
- Use of real-time availability for doctors, labs, rooms, and medical equipment.
- Multi-agent expansion where separate agents handle diagnosis support, scheduling, escalation, and monitoring.
- Addition of dashboards, analytics, and historical plan comparison for administrators.
- Improved exception handling when resources are unavailable or priorities change suddenly.
- Role-based access, audit trails, and deployment readiness for secure operational environments.

References

- Submitted High Level Design document for the Healthcare Planning Assistant Agent.
- Submitted Low Level Design document for the Healthcare Planning Assistant Agent.
- Project repository README and structure for Healthcare_Planner_Agent.
- Official documentation of CrewAI, LangChain, Streamlit, and Google Gemini API.
- General reference material on Agentic AI systems, planning agents, and workflow automation.